If the stack is protected by a mutex internally, only one thread can be running a stack member function at any one time, so the calls get nicely interleaved, while the calls to `do_something()` can run concurrently. One possible execution is then as shown in table 3.1.

Table 3.1. A possible ordering of operations on a stack from two threads

Thread A	Thread B
`if(!s.empty())`	`if(!s.empty())`
`int const value=s.top();`	`int const value=s.top();`
`s.pop();`	`s.pop();`
`do_something(value);`	`do_something(value);`

As you can see, if these are the only threads running, there's nothing in between the two calls to `top()` to modify the stack, so both threads will see the same value. Not only that, but *there are no calls to* `top()` *between the calls to* `pop()`. Consequently, one of the two values on the stack is discarded without ever having been read, whereas the other is processed twice. This is yet another race condition and far more insidious than the undefined behavior of the `empty()`/`top()` race; there's never anything obviously wrong going on, and the consequences of the bug are likely far removed from the cause, although they obviously depend on exactly what `do_something()` really does.

This calls for a more radical change to the interface, one that combines the calls to `top()` and `pop()` under the protection of the mutex. Tom Cargill[1] pointed out that a combined call can lead to issues if the copy constructor for the objects on the stack can throw an exception. This problem was dealt with fairly comprehensively from an exception-safety point of view by Herb Sutter,[2] but the potential for race conditions brings something new to the mix.